

This comprehensive guide will help you understand the flow of the program, the roles of various classes and functions, and how to execute the backtest successfully.

Table of Contents

1. [Overview](#)
2. [Script Structure](#)
3. [Detailed Code Walkthrough](#)
 - [Imports and Configuration](#)
 - [Vanguard Mapping](#)
 - [Logger Initialization](#)
 - [ETF Data Fetching](#)
 - [BondAllocationStrategy Class](#)
 - [Main Function](#)
4. [Using the Backtester](#)
 - [Prerequisites](#)
 - [Configuration](#)
 - [Running the Backtest](#)
 - [Interpreting Results](#)
5. [Best Practices and Recommendations](#)
6. [Conclusion](#)

Overview

Your script is designed to perform backtesting of a bond portfolio allocation strategy using historical data from 2011 to 2023. It leverages the Backtrader framework to simulate trading decisions and portfolio rebalancing based on economic indicators. The key components involved are:

- **DataFetcher:** Retrieves historical economic indicators and ETF price data.
- **Portfolio:** Manages allocation percentages and enforces constraints.
- **DecisionEngine:** Applies decision rules based on economic indicators to adjust allocations.
- **EconomicIndicator:** Processes and analyzes economic data.
- **BondAllocationStrategy:** Integrates all components within a Backtrader strategy to simulate trading and rebalancing.

The backtester follows an annual rebalancing strategy, adjusting the portfolio based on economic indicators each December.

Script Structure

The script is organized as follows:

1. **Imports and Configuration:** Import necessary libraries and modules.
2. **Vanguard Mapping:** Define a mapping between Vanguard mutual funds and their corresponding ETFs.
3. **Logger Initialization:** Set up logging to track the script's execution.
4. **ETF Data Fetching Function:** Define a function to fetch historical ETF data using `yfinance`.
5. **BondAllocationStrategy Class:** Define the Backtrader strategy that manages portfolio allocation and rebalancing.
6. **Main Function:** Orchestrate the backtest by setting up data feeds, initializing components, running the strategy, and analyzing results.
7. **Script Execution:** Run the `main()` function when the script is executed directly.

Detailed Code Walkthrough

Let's delve into each part of the script in detail.

Imports and Configuration

```
import logging
import sys
from datetime import datetime
from typing import Any, Dict, List, Literal, Tuple, cast

import backtrader as bt # type: ignore
import pandas as pd
import yfinance as yf # type: ignore

# Import your existing modules
from src.config import INDICATORS
from src.decision_engine import DecisionEngine
from src.indicators import EconomicIndicator
from src.portfolio import Portfolio
from src.utils import (
    collect_economic_indicators,
    initialize_data_storage_and_fetcher,
    initialize_logger,
    load_configuration,
)
```

Explanation:

- **Standard Libraries:**
 - `logging` for logging events.
 - `sys` for system-specific parameters and functions.
 - `datetime` for handling dates and times.

- `typing` for type annotations, enhancing code readability and maintainability.
- **Third-Party Libraries:**
 - `backtrader`: A popular Python library for backtesting trading strategies.
 - `pandas`: For data manipulation and analysis.
 - `yfinance`: To fetch financial data from Yahoo Finance.
- **Custom Modules:**
 - `src.config`, `src.decision_engine`, `src.indicators`, `src.portfolio`, `src.utils`: These are custom modules presumably defined within your project. They handle configuration, decision-making logic, economic indicators processing, portfolio management, and utility functions respectively.

Note: Ensure that your project structure includes these `src` modules and that they are correctly implemented to avoid import errors.

Vanguard Mapping

```
# Define the mapping from Vanguard mutual funds to ETFs
VANGUARD_MAPPING: Dict[str, str] = {
    "VBIRX": "BSV", # Vanguard Short-Term Bond ETF
    "VBLAX": "BND", # Vanguard Total Bond Market ETF
    "VTAPX": "VTIP", # Vanguard Short-Term Inflation-Protected Securities ETF
}
```

Explanation:

This dictionary maps Vanguard mutual fund tickers to their corresponding ETF tickers. It allows your strategy to use ETFs instead of mutual funds, which are generally more liquid and suitable for backtesting.

Logger Initialization

```
# Initialize the logger
logger: logging.Logger = initialize_logger()
```

Explanation:

A logger is initialized using a utility function `initialize_logger()`. Logging is crucial for monitoring the script's execution, debugging, and keeping track of significant events and errors.

Assumption: The `initialize_logger` function is defined in `src.utils` and sets up logging with appropriate handlers and formatting.

ETF Data Fetching

```
def fetch_etf_data(symbol: str, start_date: str, end_date: str) -> pd.DataFrame:
    """
    Fetch historical price data for a given ETF using yfinance.
    """
```

```

Args:
    symbol (str): The ETF ticker symbol.
    start_date (str): The start date in "YYYY-MM-DD" format.
    end_date (str): The end date in "YYYY-MM-DD" format.

Returns:
    pd.DataFrame: Historical price data with 'date' as a column.

Raises:
    ValueError: If no data is fetched for the given ETF.
"""
logger.info(f"Fetching data for ETF: {symbol}")
try:
    data: pd.DataFrame = yf.download(symbol, start=start_date, end=end_date)
    if data.empty:
        logger.error(f"No data fetched for ETF: {symbol}")
        raise ValueError(f"No data fetched for ETF: {symbol}")
    data.reset_index(inplace=True)
    data.rename(columns={"Date": "date"}, inplace=True)
    return data
except Exception as e:
    logger.error(f"Error fetching data for ETF {symbol}: {e}")
    raise

```

Explanation:

- **Purpose:** Fetches historical price data for a specified ETF within a given date range using the `yfinance` library.
- **Process:**
 1. Logs the start of data fetching for the specified ETF.
 2. Uses `yf.download()` to fetch data. The function returns a DataFrame containing Open, High, Low, Close, Adj Close, and Volume data.
 3. Checks if the fetched data is empty. If so, logs an error and raises a `ValueError`.
 4. Resets the DataFrame index to convert the 'Date' index into a column.
 5. Renames the 'Date' column to 'date' for consistency.
 6. Returns the processed DataFrame.
- **Error Handling:** Logs any exceptions that occur during data fetching and re-raises them to be handled by the calling function.

Usage: This function is used later in the `main()` function to fetch historical data for each ETF in the `VANGUARD_MAPPING`.

BondAllocationStrategy Class

```

class BondAllocationStrategy(bt.Strategy):
    """
    Backtrader Strategy for Bond Portfolio Allocation.

```

```

    This strategy integrates the existing Portfolio and DecisionEngine classes to
simulate
    investment decisions and portfolio rebalancing based on economic indicators.
    """

# Define strategy parameters with type annotations
params: Tuple[
    Tuple[str, Dict[str, float]],
    Tuple[str, int],
    Tuple[str, Dict[str, List[Dict[str, Any]]]],
] = (
    ("allocations", {"BSV": 40.0, "BND": 40.0, "VTIP": 20.0}),
    ("rebalance_frequency", 12), # Monthly frequency to check for rebalancing
    ("economic_data", {}), # Dict[str, List[Dict[str, Any]]]
)

# Define instance variables with type annotations
portfolio: Portfolio
decision_engine: DecisionEngine
economic_data: Dict[str, List[Dict[str, Any]]]
last_rebalance_year: int

def __init__(self) -> None:
    self.logger = logging.getLogger("BondAllocationStrategy")
    self.logger.debug("Initializing BondAllocationStrategy.")

    # Initialize Portfolio with initial allocations
    self.portfolio = Portfolio(allocations=self.p.allocations, logger=self.logger)
    self.logger.debug(f"Initial allocations: {self.portfolio.get_allocations()}")

    # Initialize DecisionEngine with empty indicators; will update on rebalance
    self.decision_engine = DecisionEngine(
        indicators={}, # To be populated during rebalance
        portfolio=self.portfolio,
        logger=self.logger,
    )

    # Initialize economic_data from params
    self.economic_data = self.p.economic_data

    # Track the last rebalance year to ensure annual rebalancing
    self.last_rebalance_year = 0

def next(self) -> None:
    """
    Called on each new bar. Checks if it's time to rebalance and performs
rebalancing.
    """
    current_date: datetime = self.datas[0].datetime.date(0)
    current_year: int = current_date.year
    current_month: int = current_date.month

```

```

# Check if it's December and rebalance hasn't been done for this year
if current_month == 12 and current_year != self.last_rebalance_year:
    self.logger.info(f"Rebalancing on {current_date}")
    self.perform_rebalance(current_date)
    self.last_rebalance_year = current_year

def perform_rebalance(self, rebalance_date: datetime) -> None:
    """
    Performs the portfolio rebalancing based on economic indicators up to the
    rebalance_date.

    Args:
        rebalance_date (datetime): The date to perform rebalancing.
    """
    self.logger.debug(f"Starting rebalance for date: {rebalance_date}")

    # Collect economic indicators up to rebalance_date
    indicators: Dict[str, EconomicIndicator] = self.collect_economic_indicators(
        rebalance_date
    )
    self.logger.debug(
        f"Collected {len(indicators)} economic indicators up to {rebalance_date}"
    )

    # Get the updated allocations from the current portfolio
    updated_allocations = self.portfolio.get_allocations()

    # Create a new Portfolio instance with updated allocations
    new_portfolio = Portfolio(allocations=updated_allocations, logger=self.logger)

    # Create a new DecisionEngine instance with new indicators and new portfolio
    new_decision_engine = DecisionEngine(
        indicators=indicators,
        portfolio=new_portfolio,
        logger=self.logger
    )

    # Apply decision rules to adjust portfolio allocations
    new_decision_engine.apply_decision_rules()

    # Get new updated allocations from the new portfolio
    new_updated_allocations = new_portfolio.get_allocations()

    self.logger.debug(
        f"Allocations after decision rules: {new_updated_allocations}"
    )

    # Update the strategy's portfolio and decision engine with the new instances
    self.portfolio = new_portfolio
    self.decision_engine = new_decision_engine

```

```

# Rebalance the portfolio based on the new allocations
self.rebalance_portfolio(rebalance_date)

def collect_economic_indicators(
    self, rebalance_date: datetime
) -> Dict[str, EconomicIndicator]:
    """
    Collects and initializes EconomicIndicator instances up to the rebalance_date.

    Args:
        rebalance_date (datetime): The date up to which data should be collected.

    Returns:
        Dict[str, EconomicIndicator]: A dictionary of EconomicIndicator instances.
    """
    indicators: Dict[str, EconomicIndicator] = {}
    for key, data in self.economic_data.items():
        # Filter data up to rebalance_date
        filtered_data: List[Dict[str, Any]] = [
            entry
            for entry in data
            if datetime.strptime(entry["date"], "%Y-%m-%d").date() <= rebalance_date
        ]
        # Find the corresponding IndicatorConfig
        indicator_config = next(
            (ind for ind in INDICATORS if ind.internal_key == key), None
        )
        if indicator_config:
            indicator = EconomicIndicator(
                config=indicator_config, data=filtered_data, logger=self.logger
            )
            indicators[key] = indicator
        else:
            self.logger.warning(f"No IndicatorConfig found for key: {key}")
    return indicators

def rebalance_portfolio(self, rebalance_date: datetime) -> None:
    """
    Rebalances the portfolio based on the current allocations.

    Args:
        rebalance_date (datetime): The date on which rebalancing is performed.
    """
    total_value: float = self.broker.getvalue()
    self.logger.debug(f"Total portfolio value: ${total_value:,.2f}")

    for symbol, percentage in self.portfolio.get_allocations().items():
        data: bt.feeds.PandasData = self.getdatabyname(symbol)
        current_price: float = data.close[0]
        target_value: float = total_value * (percentage / 100.0)
        current_position: float = self.getposition(data).size
        current_value: float = current_position * current_price

```

```

value_difference: float = target_value - current_value
number_of_shares: int = int(value_difference / current_price)

if number_of_shares > 0:
    self.logger.debug(
        f"Buying {number_of_shares} shares of {symbol} to reach target
allocation."
    )
    self.buy(data=data, size=number_of_shares)
elif number_of_shares < 0:
    self.logger.debug(
        f"Selling {abs(number_of_shares)} shares of {symbol} to reach target
allocation."
    )
    self.sell(data=data, size=abs(number_of_shares))
else:
    self.logger.debug(f"No action needed for {symbol}.")

def notify_order(self, order: bt.Order) -> None:
    """
    Called when an order's status changes.

    Args:
        order (bt.Order): The order object.
    """
    if order.status in [order.Submitted, order.Accepted]:
        # Order has been submitted/accepted but not yet completed
        return

    if order.status in [order.Completed]:
        if order.isbuy():
            self.logger.info(
                f"BUY EXECUTED: {order.executed.size} shares of {order.data._name} at
{order.executed.price:.2f}"
            )
        elif order.issell():
            self.logger.info(
                f"SELL EXECUTED: {order.executed.size} shares of {order.data._name}
at {order.executed.price:.2f}"
            )
        # Assuming 'bar_executed' is used elsewhere; initialize if necessary
        self.bar_executed = len(self)

    elif order.status in [order.Canceled, order.Margin, order.Rejected]:
        self.logger.warning(
            f"Order Canceled/Margin/Rejected for {order.data._name}"
        )

def notify_trade(self, trade: bt.Trade) -> None:
    """
    Called when a trade is updated.

```

```

    Args:
        trade (bt.Trade): The trade object.
    """
    if not trade.isclosed:
        return

    self.logger.info(
        f"TRADE PROFIT, GROSS {trade.pnl:.2f}, NET {trade.pnlcomm:.2f}"
    )

```

Explanation:

The `BondAllocationStrategy` class defines the trading strategy to be used by Backtrader. It manages portfolio allocations, applies decision rules based on economic indicators, and rebalances the portfolio annually.

Key Components:

1. Parameters (`params`):

- `allocations`: Initial allocation percentages for each ETF.
- `rebalance_frequency`: Frequency (in months) to check for rebalancing. Set to 12 for annual rebalancing.
- `economic_data`: A dictionary containing economic indicators data.

2. Instance Variables:

- `portfolio`: An instance of the `Portfolio` class managing current allocations.
- `decision_engine`: An instance of the `DecisionEngine` class applying decision rules.
- `economic_data`: Economic indicators data passed from the main function.
- `last_rebalance_year`: Tracks the last year when rebalancing occurred to ensure annual rebalancing.

3. `__init__` Method:

- Initializes the logger specific to the strategy.
- Initializes the `Portfolio` with initial allocations.
- Initializes the `DecisionEngine` with empty indicators (to be populated during rebalancing).
- Sets `economic_data` and initializes `last_rebalance_year` to 0.

4. `next` Method:

- Called on each new data bar (typically daily data).
- Checks if the current month is December and if rebalancing hasn't been done for the current year.
- If conditions are met, it calls `perform_rebalance` and updates `last_rebalance_year`.

5. `perform_rebalance` Method:

- Collects economic indicators up to the rebalance date.
- Retrieves current allocations from the portfolio.

- Creates a new `Portfolio` and `DecisionEngine` instance with updated allocations and indicators.
- Applies decision rules to adjust allocations.
- Updates the strategy's `portfolio` and `decision_engine` with the new instances.
- Calls `rebalance_portfolio` to execute trades based on new allocations.

6. **collect_economic_indicators Method:**

- Filters economic data up to the rebalance date.
- Initializes `EconomicIndicator` instances for each indicator.
- Returns a dictionary of these indicators.

7. **rebalance_portfolio Method:**

- Calculates the total portfolio value.
- Iterates over each ETF and calculates the target value based on allocation percentages.
- Determines the number of shares to buy or sell to reach the target allocation.
- Executes buy/sell orders accordingly.

8. **notify_order Method:**

- Monitors order status changes.
- Logs execution details for buy/sell orders.
- Handles order cancellations, margin calls, and rejections.

9. **notify_trade Method:**

- Monitors trade updates.
- Logs profit or loss details once a trade is closed.

Rebalancing Logic Update:

The `perform_rebalance` method has been revised to follow a specific pattern:

1. **Collect Economic Indicators:** Gathers relevant economic data up to the rebalance date.
2. **Retrieve Current Allocations:** Gets the current portfolio allocations.
3. **Create New Portfolio Instance:** Initializes a new `Portfolio` with current allocations.
4. **Create New DecisionEngine Instance:** Initializes a new `DecisionEngine` with the new portfolio and collected indicators.
5. **Apply Decision Rules:** Adjusts allocations based on decision rules.
6. **Retrieve Updated Allocations:** Gets the updated allocations from the new portfolio.
7. **Update Strategy's Instances:** Replaces the old `portfolio` and `decision_engine` with the new instances.
8. **Rebalance Portfolio:** Executes trades to align with the new allocations.

This approach ensures that each rebalance is based on fresh instances, avoiding unintended side effects from modifying existing instances.

Main Function

```

def main() -> None:
    """
    Main function to set up and run the backtest.
    """
    logger.info("Starting backtest.")

    # Define configuration file path
    config_path: str = "config/config.yaml"

    # Define required configuration fields
    required_config_fields: List[str] = [
        "storage.fred_file_path",
        "storage.recommendation_file_path",
        "api.fred_api_key",
    ]

    # Load configuration
    try:
        config: Dict[str, Any] = load_configuration(
            config_path, logger, required_config_fields
        )
    except Exception as e:
        logger.error(f"Failed to load configuration: {e}")
        sys.exit(1)

    # Initialize data storage and fetcher
    try:
        fetcher: Any = initialize_data_storage_and_fetcher(config, logger)
    except Exception as e:
        logger.error(f"Failed to initialize data storage and fetcher: {e}")
        sys.exit(1)

    # Initialize Portfolio with existing allocations
    try:
        portfolio: Portfolio = Portfolio(
            allocations={"BSV": 40.0, "BND": 40.0, "VTIP": 20.0}, logger=logger
        )
        logger.info(f"Initial Portfolio Allocations: {portfolio.get_allocations()}")
    except Exception as e:
        logger.critical(f"Error initializing Portfolio: {e}")
        sys.exit(1)

    # Fetch economic indicators
    try:
        indicators: Dict[str, EconomicIndicator] = collect_economic_indicators(
            fetcher, config.get("indicators", INDICATORS), logger
        )
    except Exception as e:
        logger.error(f"Failed to collect economic indicators: {e}")
        sys.exit(1)

```

```

# Prepare economic_data dictionary for the strategy
economic_data: Dict[str, List[Dict[str, Any]]] = {}
for indicator in indicators.values():
    if isinstance(indicator.data, pd.DataFrame):
        economic_data[indicator.internal_key] = cast(
            List[Dict[str, Any]], indicator.data.to_dict(orient="records")
        )
    else:
        economic_data[indicator.internal_key] = cast(
            List[Dict[str, Any]], indicator.data
        )

# Fetch historical ETF data (2011-2023)
etf_start_date: str = "2011-01-01"
etf_end_date: str = "2023-12-31"
bond_data_feeds: Dict[str, bt.feeds.PandasData] = {}

for fund, etf in VANGUARD_MAPPING.items():
    try:
        df: pd.DataFrame = fetch_etf_data(
            symbol=etf, start_date=etf_start_date, end_date=etf_end_date
        )

        # Rename necessary columns to lowercase for consistency
        df.rename(
            columns={
                "Date": "date",
                "Open": "open",
                "High": "high",
                "Low": "low",
                "Close": "close",
            },
            inplace=True,
        )

        # Log the columns after renaming for verification
        logger.debug(f"Columns after renaming for {etf}: {df.columns.tolist()}")

        # Validate required columns
        required_columns: set = {"date", "open", "high", "low", "close"}
        missing_columns: set = required_columns - set(df.columns)
        if missing_columns:
            logger.error(
                f"Data for ETF {etf} is missing required columns: {missing_columns}"
            )
            sys.exit(1)

        # Proceed to set volume and openinterest
        df["volume"] = 0 # Volume is not used for bonds
        df["openinterest"] = 0
        df.set_index("date", inplace=True)

```

```

# Ensure no missing data
if df.isnull().values.any():
    logger.warning(
        f"Missing data detected for ETF {etf}. Applying forward fill."
    )
    df = df.ffill()
    if df.isnull().values.any():
        logger.warning(
            f"Data for ETF {etf} still contains missing values after forward
fill."
        )

data_feed: bt.feeds.PandasData = bt.feeds.PandasData(
    dataname=df,
    name=etf,
    fromdate=datetime(2011, 1, 1),
    todate=datetime(2023, 12, 31),
)
bond_data_feeds[etf] = data_feed
logger.info(f"Data for ETF {etf} fetched and prepared.")
except Exception as e:
    logger.error(f"Failed to fetch data for ETF {etf}: {e}")
    sys.exit(1)

# Initialize Backtrader Cerebro engine
cerebro: bt.Cerebro = bt.Cerebro()
cerebro.addstrategy(BondAllocationStrategy, economic_data=economic_data)

# Add data feeds to Cerebro
for etf, data_feed in bond_data_feeds.items():
    cerebro.adddata(data_feed, name=etf)
    logger.debug(f"Added data feed for ETF {etf}.")

# Set initial capital
initial_capital: float = 100000.0
cerebro.broker.setcash(initial_capital)
logger.info(f"Initial capital set to ${initial_capital:,.2f}")

# Set commission - assuming 0.1% commission per trade
cerebro.broker.setcommission(commission=0.001)
logger.info("Commission set to 0.1% per trade.")

# Add analyzers if needed (e.g., Sharpe Ratio, Drawdown)
cerebro.addanalyzer(bt.analyzers.SharpeRatio, _name="sharpe")
cerebro.addanalyzer(bt.analyzers.DrawDown, _name="drawdown")

# Run the backtest
logger.info("Running backtest...")
try:
    results: List[BondAllocationStrategy] = cast(
        List[BondAllocationStrategy], cerebro.run()
    )

```

```

except Exception as e:
    logger.critical(f"Backtest failed: {e}")
    sys.exit(1)

# Get the strategy instance
strategy: BondAllocationStrategy = results[0]

# Print final portfolio value
final_value: float = cerebro.broker.getvalue()
logger.info(f"Final Portfolio Value: ${final_value:,.2f}")

# Print analyzers' results
sharpe: Dict[str, Any] = cast(
    Dict[str, Any], strategy.analyzers.sharpe.get_analysis()
)
drawdown: Dict[str, Any] = cast(
    Dict[str, Any], strategy.analyzers.drawdown.get_analysis()
)
logger.info(f"Sharpe Ratio: {sharpe.get('sharperatio', 'N/A')}")
logger.info(f"Max Drawdown: {drawdown.get('max', 'N/A')}%")

# Plot the results
cerebro.plot(style="candlestick")

```

Explanation:

The `main()` function orchestrates the entire backtesting process. Here's a breakdown of each step:

1. Initialization:

- Logs the start of the backtest.
- Defines the path to the configuration file (`config/config.yaml`).
- Specifies required configuration fields to ensure all necessary parameters are provided.

2. Configuration Loading:

- Attempts to load the configuration using `load_configuration()`. If it fails, logs the error and exits.

3. Data Storage and Fetcher Initialization:

- Initializes data storage and the data fetcher using `initialize_data_storage_and_fetcher()`. This setup is crucial for retrieving economic indicators and other necessary data.

4. Portfolio Initialization:

- Initializes the `Portfolio` with initial allocations as defined in `VANGUARD_MAPPING` (e.g., 40% BSV, 40% BND, 20% VTIP).
- Logs the initial allocations.

5. Economic Indicators Fetching:

- Collects economic indicators using `collect_economic_indicators()`. These indicators will influence portfolio allocation decisions during rebalancing.

6. Preparing Economic Data for the Strategy:

- Transforms the collected `EconomicIndicator` data into a dictionary format suitable for passing to the strategy. It ensures that the data is in a list of dictionaries format, facilitating easy access during rebalancing.

7. Fetching Historical ETF Data (2011-2023):

- Defines the start and end dates for historical data fetching.
- Iterates over each ETF in `VANGUARD_MAPPING` and fetches its historical data using `fetch_etf_data()`.
- Processes the fetched data:
 - Renames columns to lowercase for consistency.
 - Validates that all required columns (`date`, `open`, `high`, `low`, `close`) are present.
 - Sets `volume` and `openinterest` to 0, as they are not used for bonds.
 - Sets the 'date' column as the index.
 - Checks for missing data and applies forward fill (`ffill()`) to handle any gaps.
- Creates a Backtrader data feed (`bt.feeds.PandasData`) for each ETF and adds it to `bond_data_feeds`.
- Logs the successful preparation of each ETF's data.

8. Backtrader Cerebro Engine Setup:

- Initializes Backtrader's Cerebro engine.
- Adds the `BondAllocationStrategy` to Cerebro, passing the prepared `economic_data`.

9. Adding Data Feeds to Cerebro:

- Iterates over each ETF's data feed and adds it to Cerebro using `cerebro.adddata()`.
- Logs the addition of each data feed.

10. Broker Configuration:

- Sets the initial capital (e.g., \$100,000).
- Sets the commission rate (e.g., 0.1% per trade).
- Logs both configurations.

11. Adding Analyzers:

- Adds Sharpe Ratio and DrawDown analyzers to evaluate the strategy's performance.
- These analyzers provide insights into the risk-adjusted returns and the maximum portfolio drawdown.

12. Running the Backtest:

- Logs the start of the backtest.
- Executes the strategy using `cerebro.run()`.
- Catches and logs any exceptions that occur during the backtest.

13. Post-Backtest Analysis:

- Retrieves the final portfolio value using `cerebro.broker.getvalue()`.
- Logs the final portfolio value.

- Retrieves and logs the Sharpe Ratio and Maximum Drawdown from the analyzers.

14. Plotting Results:

- Plots the backtest results using `cerebro.plot()`. The plot will display the portfolio's performance over time with candlestick charts for the ETFs.

Assumptions:

- **Configuration File (`config/config.yaml`):** Contains necessary configurations such as file paths and API keys.
- **Custom Modules:**
 - `load_configuration()` : Loads and validates the configuration file.
 - `initialize_data_storage_and_fetcher()` : Sets up data storage mechanisms and data fetchers.
 - `collect_economic_indicators()` : Collects and processes economic indicators based on configurations.

Ensure these modules are correctly implemented and accessible in your project structure.

Using the Backtester

Now that we've walked through the code, let's discuss how to set up and use the backtester effectively.

Prerequisites

1. Python Environment:

- **Version:** Python 3.8 or higher.
- **Virtual Environment:** It's recommended to use a virtual environment to manage dependencies.

2. Dependencies:

- **Third-Party Libraries:**

- `backtrader`
- `pandas`
- `yfinance`
- Any other dependencies specified in your project configuration or `requirements.txt`.

- **Installation:**

```
pip install backtrader pandas yfinance
```

- **Note:** If you have a `requirements.txt`, install dependencies using:

```
pip install -r requirements.txt
```

3. Project Structure:

- Ensure that your project directory contains the `src` folder with all the necessary modules (`config.py`, `decision_engine.py`, `indicators.py`, `portfolio.py`, `utils.py`, etc.).
- Place the `backtester.py` script (the code you've provided) in an appropriate location, e.g., in the `src` folder or at the root.

4. Configuration File (`config/config.yaml`):

- Create a YAML configuration file with the required fields:

```
storage:
  fred_file_path: "path/to/fred/data.csv"
  recommendation_file_path: "path/to/recommendations.csv"
api:
  fred_api_key: "YOUR_FRED_API_KEY"
indicators:
  # Define your indicators here as per INDICATORS in src.config
```

◦ Fields Explained:

- `storage.fred_file_path`: Path to the FRED (Federal Reserve Economic Data) file.
- `storage.recommendation_file_path`: Path to any recommendation data files.
- `api.fred_api_key`: Your API key for accessing FRED data.
- `indicators`: Definitions of economic indicators used in your strategy.

5. Economic Indicators Data:

- Ensure that the economic indicators required by your strategy are available and correctly formatted.

6. Vanguard Mapping:

- Verify that the ETFs specified in `VANGUARD_MAPPING` are correct and active.

Configuration

1. Edit `config/config.yaml`:

- Populate the configuration file with accurate paths and API keys.
- Define any additional settings required by your economic indicators.

2. Verify Data Sources:

- Ensure that the paths to data files (e.g., FRED data) are correct.
- Verify that the API keys are valid and have the necessary permissions.

3. Adjust Allocations (Optional):

- If you wish to change the initial allocations or rebalance frequency, modify the `params` in the `BondAllocationStrategy` class accordingly.

Running the Backtest

1. Navigate to Project Directory:

- Open your terminal or command prompt and navigate to the project's root directory.

2. Execute the Backtester:

- Run the `backtester.py` script using Python:

```
python src/backtester.py
```

- **Note:** Adjust the path if `backtester.py` is located elsewhere.

3. Monitor Execution:

- The script will log its progress, including data fetching, portfolio initialization, rebalancing actions, and trade executions.
- Ensure that there are no errors during execution. If any errors occur, refer to the logs for debugging.

4. View Results:

- Upon completion, the script will display:
 - Final Portfolio Value.
 - Sharpe Ratio.
 - Maximum Drawdown.
- A plot will also be generated showing the portfolio's performance over time with candlestick charts for the ETFs.

Interpreting Results

1. Final Portfolio Value:

- Indicates the portfolio's worth at the end of the backtest period.
- Compare this with the initial capital to assess overall performance.

2. Sharpe Ratio:

- Measures risk-adjusted return.
- A higher Sharpe Ratio indicates better performance relative to the risk taken.

3. Maximum Drawdown:

- Represents the largest peak-to-trough decline in the portfolio's value.
- Helps assess the strategy's risk during adverse market conditions.

4. Performance Plot:

- Visualizes the portfolio's growth over time.
- Candlestick charts provide insights into the price movements of the underlying ETFs.

Note: It's essential to analyze these metrics collectively to understand the strategy's effectiveness and risk profile.

Best Practices and Recommendations

1. Modular Code Structure:

- Maintain a clean and modular codebase. Separate concerns by organizing related functionalities into distinct modules.
- This enhances readability, maintainability, and scalability.

2. **Logging:**

- Utilize logging extensively to monitor the script's execution and debug issues.
- Differentiate between log levels (INFO, DEBUG, WARNING, ERROR, CRITICAL) based on the severity of events.

3. **Error Handling:**

- Implement robust error handling to catch and manage exceptions gracefully.
- Provide meaningful error messages to facilitate troubleshooting.

4. **Configuration Management:**

- Use configuration files to manage settings, making it easier to adjust parameters without modifying the code.
- Validate configurations to ensure all required fields are present and correctly formatted.

5. **Data Validation:**

- Always verify the integrity and completeness of data before using it in calculations.
- Handle missing or anomalous data appropriately to avoid skewed results.

6. **Version Control:**

- Use version control systems like Git to track changes, collaborate with others, and maintain code history.

7. **Testing:**

- Develop unit tests for critical components to ensure they function as expected.
- Perform integration tests to verify that different parts of the system work together seamlessly.

8. **Documentation:**

- Document your code thoroughly, including docstrings for classes and functions.
- Maintain a README file that provides an overview of the project, setup instructions, and usage guidelines.

9. **Performance Optimization:**

- For large datasets, consider optimizing data processing steps to enhance performance.
- Utilize efficient data structures and algorithms to minimize computational overhead.

10. **Rebalancing Strategy:**

- While annual rebalancing is implemented, consider testing different rebalancing frequencies to assess their impact on performance.
- Explore dynamic rebalancing based on market conditions or other triggers.

Conclusion

Your backtesting script is a comprehensive tool designed to evaluate a bond portfolio allocation strategy using historical data and economic indicators. By following this walkthrough, you should have a clear understanding of each component's role and how they interact within the system. Proper configuration, data management, and adherence to best practices will ensure that your backtests are reliable and provide meaningful insights into your investment strategy's performance.

Next Steps:

1. Validate the Backtest:

- Run the backtest and verify that the results align with your expectations.
- Check the logs for any anomalies or unexpected behaviors.

2. Enhance the Strategy:

- Experiment with different economic indicators and decision rules to optimize portfolio performance.
- Incorporate additional analyzers or performance metrics to gain deeper insights.

3. Expand the Backtester:

- Add support for more asset classes or ETFs to diversify the portfolio.
- Implement more sophisticated trading rules or machine learning models for decision-making.